

LAVISH GAMBHIR

1024030260

3C21

Q1.

```
public class ExceptionHierarchyDemo {
    public static void main(String[] args) {
        try {
            int a = 10 / 0;
        } catch (ArithmeticException e) {
            System.out.println("ArithmeticException: " + e.getMessage());
        }

        try {
            String s = null;
            System.out.println(s.length());
        } catch (NullPointerException e) {
            System.out.println("NullPointerException: object was null");
        }

        try {
            int[] arr = new int[3];
            arr[5] = 10;
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("ArrayIndexOutOfBoundsException: " + e.getMessage());
        }

        try {
            int num = Integer.parseInt("abc");
        } catch (NumberFormatException e) {
            System.out.println("NumberFormatException: " + e.getMessage());
        }

        System.out.println("Throwable -> Exception -> RuntimeException -> ArithmeticException,
        NullPointerException, ArrayIndexOutOfBoundsException, NumberFormatException");
    }
}
```

Q2.

```
public class NestedTryCatchDemo {
    public static void main(String[] args) {
        try {
            System.out.println("Outer try block started");
            int[] arr = {1, 2, 3};
        }
    }
}
```

```

try {
    System.out.println("Inner try block started");
    int result = 10 / 0;
    System.out.println(result);
} catch (ArithmeticException e) {
    System.out.println("Inner catch: ArithmeticException - " + e.getMessage());
}

System.out.println(arr[5]);

} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Outer catch: ArrayIndexOutOfBoundsException - " + e.getMessage());
} finally {
    System.out.println("Outer finally block executed");
}
}

```

Q3.

```

import java.util.Scanner;

public class SimpleCalculator {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter first number: ");
            double num1 = Double.parseDouble(sc.nextLine());

            System.out.print("Enter operator (+, -, *, /): ");
            String operator = sc.nextLine();

            System.out.print("Enter second number: ");
            double num2 = Double.parseDouble(sc.nextLine());

            double result = calculate(num1, num2, operator);
            System.out.println("Result: " + result);

        } catch (NumberFormatException e) {
            System.out.println("Error: Invalid numeric input. Please enter valid numbers.");
        } catch (ArithmeticException e) {
            System.out.println("Error: " + e.getMessage());
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        } finally {
            sc.close();
        }
    }

    static double calculate(double a, double b, String op) {
        switch (op) {

```

```

        case "+": return a + b;
        case "-": return a - b;
        case "*": return a * b;
        case "/":
            if (b == 0) {
                throw new ArithmeticException("Division by zero is not allowed.");
            }
            return a / b;
        default:
            throw new IllegalArgumentException("Invalid operator entered: " + op);
    }
}
}

```

Q4.

```

import java.util.Scanner;

class InvalidMarksException extends Exception {
    public InvalidMarksException(String message) {
        super(message);
    }
}

public class StudentResultProcessing {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int numSubjects = 5;
        int[] marks = new int[numSubjects];
        boolean allValid = true;

        for (int i = 0; i < numSubjects; i++) {
            try {
                System.out.print("Enter marks for subject " + (i + 1) + ": ");
                int m = Integer.parseInt(sc.nextLine());
                if (m < 0 || m > 100) {
                    throw new InvalidMarksException("Marks must be between 0 and 100. You entered: " + m);
                }
                marks[i] = m;
            } catch (InvalidMarksException e) {
                System.out.println("Error: " + e.getMessage());
                allValid = false;
            } catch (NumberFormatException e) {
                System.out.println("Error: Please enter a valid number.");
                allValid = false;
            }
        }

        if (allValid) {
            int total = 0;
            for (int m : marks) total += m;
        }
    }
}

```

```
double percentage = total / (double) numSubjects;
char grade = getGrade(percentage);

System.out.println("Total: " + total);
System.out.println("Percentage: " + percentage + "%");
System.out.println("Grade: " + grade);
} else {
    System.out.println("Result cannot be calculated because some marks were invalid.");
}

sc.close();
}

static char getGrade(double percentage) {
    if (percentage >= 90) return 'A';
    else if (percentage >= 75) return 'B';
    else if (percentage >= 60) return 'C';
    else if (percentage >= 40) return 'D';
    else return 'F';
}
}
```

Q5.

```
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) { super(message); }
}

class InvalidAmountException extends Exception {
    public InvalidAmountException(String message) { super(message); }
}

class AccountNotFoundException extends Exception {
    public AccountNotFoundException(String message) { super(message); }
}

class BankAccount {
    private double balance;

    public BankAccount(double balance) { this.balance = balance; }

    public double getBalance() { return balance; }

    public void deposit(double amount) throws InvalidAmountException {
        if (amount <= 0) {
            throw new InvalidAmountException("Deposit amount must be positive.");
        }
        balance += amount;
    }

    public void withdraw(double amount) throws InvalidAmountException, InsufficientBalanceException {
        if (amount <= 0) {
            throw new InvalidAmountException("Withdrawal amount must be positive.");
        }
        if (amount > balance) {
            throw new InsufficientBalanceException("Insufficient balance. Available: " + balance);
        }
        balance -= amount;
    }
}

class Bank {
    private Map<String, BankAccount> accounts = new HashMap<>();

    public void addAccount(String accNo, double balance) {
        accounts.put(accNo, new BankAccount(balance));
    }

    public BankAccount getAccount(String accNo) throws AccountNotFoundException {
        BankAccount acc = accounts.get(accNo);
    }
}
```

```

        if (acc == null) {
            throw new AccountNotFoundException("Account not found: " + accNo);
        }
        return acc;
    }

    public void performWithdrawal(String accNo, double amount)
        throws AccountNotFoundException, InvalidAmountException, InsufficientBalanceException {
        BankAccount acc = getAccount(accNo);
        acc.withdraw(amount);
    }
}

public class SimpleBankingApp {
    public static void main(String[] args) {
        Bank bank = new Bank();
        bank.addAccount("ACC101", 5000);
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter account number: ");
            String accNo = sc.nextLine();

            BankAccount acc = bank.getAccount(accNo);

            System.out.print("Enter amount to deposit: ");
            double depositAmt = Double.parseDouble(sc.nextLine());
            acc.deposit(depositAmt);
            System.out.println("Deposit successful. Balance: " + acc.getBalance());

            System.out.print("Enter amount to withdraw: ");
            double withdrawAmt = Double.parseDouble(sc.nextLine());
            bank.performWithdrawal(accNo, withdrawAmt);
            System.out.println("Withdrawal successful. Balance: " + acc.getBalance());

        } catch (AccountNotFoundException | InvalidAmountException | InsufficientBalanceException e) {
            System.out.println("Error: " + e.getMessage());
        } catch (NumberFormatException e) {
            System.out.println("Error: Please enter a valid amount.");
        } finally {
            sc.close();
        }
    }
}

```

Q6.

```

import java.util.Scanner;

class InvalidUsernameException extends Exception {
    public InvalidUsernameException(String message) { super(message); }
}

```

```

class InvalidPasswordException extends Exception {
    public InvalidPasswordException(String message) { super(message); }
}

class AccountLockedException extends Exception {
    public AccountLockedException(String message) { super(message); }
}

public class LoginSystem {
    static final String CORRECT_USERNAME = "admin";
    static final String CORRECT_PASSWORD = "pass123";
    static final int MAX_ATTEMPTS = 3;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int attempts = 0;
        boolean loggedIn = false;
        boolean locked = false;

        while (attempts < MAX_ATTEMPTS && !loggedIn && !locked) {
            try {
                System.out.print("Enter username: ");
                String username = sc.nextLine();

                System.out.print("Enter password: ");
                String password = sc.nextLine();

                validateLogin(username, password);
                loggedIn = true;
                System.out.println("Login successful. Welcome, " + username + "!");
            } catch (InvalidUsernameException e) {
                attempts++;
                System.out.println("Error: " + e.getMessage() + " (Attempt " + attempts + "/" + MAX_ATTEMPTS + ")");
            } catch (InvalidPasswordException e) {
                attempts++;
                System.out.println("Error: " + e.getMessage() + " (Attempt " + attempts + "/" + MAX_ATTEMPTS + ")");
            } finally {
                if (!loggedIn && attempts >= MAX_ATTEMPTS) {
                    locked = true;
                }
            }
        }

        if (locked) {
            try {
                throw new AccountLockedException("Account locked. Maximum login attempts exceeded.");
            } catch (AccountLockedException e) {
                System.out.println("Error: " + e.getMessage());
            }
        }
    }
}

```

```

        sc.close();
    }

    static void validateLogin(String username, String password)
        throws InvalidUsernameException, InvalidPasswordException {
        if (!username.equals(CORRECT_USERNAME)) {
            throw new InvalidUsernameException("Username does not exist.");
        }
        if (!password.equals(CORRECT_PASSWORD)) {
            throw new InvalidPasswordException("Incorrect password.");
        }
    }
}

```

Q7.

```

import java.util.HashMap;
import java.util.Map;

class ApplicationException extends Exception {
    public ApplicationException(String message) { super(message); }
}

class ProductException extends ApplicationException {
    public ProductException(String message) { super(message); }
}

class ProductNotFoundException extends ProductException {
    public ProductNotFoundException(String message) { super(message); }
}

class OutOfStockException extends ProductException {
    public OutOfStockException(String message) { super(message); }
}

class PaymentException extends ApplicationException {
    public PaymentException(String message) { super(message); }
}

class InvalidPaymentException extends PaymentException {
    public InvalidPaymentException(String message) { super(message); }
}

class InsufficientFundsException extends PaymentException {
    public InsufficientFundsException(String message) { super(message); }
}

class OrderException extends ApplicationException {
    public OrderException(String message) { super(message); }
}

class EmptyCartException extends OrderException {
    public EmptyCartException(String message) { super(message); }
}

class Product {

```

```

String name; double price; int stock;
Product(String name, double price, int stock) {
    this.name = name; this.price = price; this.stock = stock;
}
}

class ShoppingCart {
    Map<String, Product> catalog = new HashMap<>();
    Map<String, Integer> cart = new HashMap<>();

    void addProductToCatalog(String name, double price, int stock) {
        catalog.put(name, new Product(name, price, stock));
    }

    Product searchProduct(String name) throws ProductNotFoundException {
        Product p = catalog.get(name);
        if (p == null) throw new ProductNotFoundException("Product not found: " + name);
        return p;
    }

    void addToCart(String name, int qty) throws ProductNotFoundException, OutOfStockException {
        Product p = searchProduct(name);
        if (p.stock < qty) {
            throw new OutOfStockException("Not enough stock for " + name + ". Available: " + p.stock);
        }
        cart.put(name, cart.getOrDefault(name, 0) + qty);
    }

    void removeFromCart(String name) throws ProductNotFoundException {
        if (!cart.containsKey(name)) throw new ProductNotFoundException("Product not in cart: " + name);
        cart.remove(name);
    }

    double getCartTotal() throws EmptyCartException {
        if (cart.isEmpty()) throw new EmptyCartException("Cart is empty. Add items before checkout.");
        double total = 0;
        for (Map.Entry<String, Integer> entry : cart.entrySet()) {
            total += catalog.get(entry.getKey()).price * entry.getValue();
        }
        return total;
    }

    void makePayment(double amountPaid)
        throws EmptyCartException, InvalidPaymentException, InsufficientFundsException {
        double total = getCartTotal();
        if (amountPaid <= 0) throw new InvalidPaymentException("Payment amount must be positive.");
        if (amountPaid < total) {
            throw new InsufficientFundsException("Insufficient funds. Required: " + total + ", Paid: " + amountPaid);
        }
        System.out.println("Payment successful. Change returned: " + (amountPaid - total));
        cart.clear();
    }
}

```

```
}  
}
```

```
public class ShoppingCartApp {  
    public static void main(String[] args) {  
        ShoppingCart cart = new ShoppingCart();  
        cart.addProductToCatalog("Laptop", 50000, 2);  
        cart.addProductToCatalog("Mouse", 500, 0);  
  
        try {  
            cart.addToCart("Laptop", 1);  
            System.out.println("Laptop added to cart.");  
            cart.addToCart("Mouse", 1);  
        } catch (ProductNotFoundException e) {  
            System.out.println("Product error: " + e.getMessage());  
        } catch (OutOfStockException e) {  
            System.out.println("Stock error: " + e.getMessage());  
        }  
  
        try {  
            System.out.println("Cart total: " + cart.getCartTotal());  
            cart.makePayment(20000);  
        } catch (EmptyCartException | InvalidPaymentException | InsufficientFundsException e) {  
            System.out.println("Order/Payment error: " + e.getMessage());  
        }  
  
        try {  
            cart.makePayment(50000);  
        } catch (ApplicationException e) {  
            System.out.println("Error during payment: " + e.getMessage());  
        }  
    }  
}
```